

# Crystal Portfolio Hero Overview Simplification Report

## Executive summary

Enabled connector used for this audit: **GitHub**.

I audited both the current default branch ( `master` ) and the active transition-work branch behind open PR #381 ( `codex/audit-crystal-transition-animation-code` ), because PR #381 is where the current hero→overview phase experiments live. That PR adds a new explicit phase system over five files with **1,021 additions** and **51 deletions**, on top of the existing camera/scene transition logic rather than replacing it.

1

The core problem is architectural, not “one more timing bug.” In the current transition branch, `useUnifiedAnimationController` owns a mutable `transitionPhase` state and exposes `setTransitionPhase`, while `UnifiedCrystalScene` moves phases forward from swap/explosion events and `UnifiedCameraController` still carries its own long-lived ownership refs ( `fractureTiltActiveRef`, `heroExplosionTransitionRef`, `authoritativeHeroToOverviewTransitionRef`, handoff refs, etc.). That means the codebase now has **two overlapping lifecycles** for the same cinematic: a controller phase machine and an older camera/scene ownership machine. They are related, but not identical, which is exactly the kind of structure that produces freezes, stale ownership, and “phase says complete but visuals are still held” behavior. 2 3 4 5

The fastest credible path forward is **not** to keep patching PR #381. The right move is to start a **fresh implementation branch from the last stable visual baseline**, keep PR #381 open only as a reference branch, and introduce one small dedicated **HeroOverviewCinematicTransition** runtime that becomes the single source of truth for the hero→overview cinematic. Camera and scene should each have exactly one cinematic writer branch gated by this runtime; all legacy branches must early-return while it is active. That preserves the rest of the app, preserves the current control panel workflow, and sharply reduces debugging surface area. This recommendation also matches the repo’s own earlier low-risk direction in PR #342, which explicitly preferred a surgical masking/transition fix over a broad new state machine. 6

The desktop/mobile JSON workflow is already strong enough to support this. `CrystalControls` already clones config, builds a versioned layout payload, and imports it back into runtime config; `useLayoutConfig` already selects desktop vs mobile JSON and validates it through `parseLayout`; `App.jsx` already rebuilds the runtime animation config from edited config. The missing piece is simply that the layout schema does **not yet** define a backward-compatible hero→overview cinematic tuning block. That is an integration gap, not a reason to rewrite the app. 7 8 9 10

## Scope and evidence

This audit is based primarily on repository code and PR metadata, with official documentation used only where it materially supports the design recommendation. I inspected the current application composition in `App.jsx`, `Fixed3DCanvas.jsx`, `MasterAnimationCoordinator.jsx`, `useUnifiedAnimationController.js`, `UnifiedCrystalScene.jsx`, `UnifiedCameraController.jsx`, the layout export/import path (`CrystalControls.jsx`, `useLayoutConfig.js`, `parseLayout.js`, `desktop.json`, `mobile.json`), and relevant PRs #342, #365, #379, and #381. 10 11 5 2 3 4 7 8 9 12 13 6 14 15 1

A practical limitation: several very large files were truncated in the connector response. Where lower-file behavior mattered, I relied on the matching open-PR metadata and the visible code structure rather than inventing certainty. The architectural conclusions are still high-confidence because the critical ownership/state relationships are visible in the fetched portions and PR descriptions. 1

One more important procedural finding: the repository's current `package.json` exposes `dev`, `build`, and `preview`, but **no test or lint scripts**. That means the real validation path for this refactor is “small commits + `npm run build` + browser QA,” not “trust CI.” Some PR descriptions mention other checks, but those scripts are not present in the current repo manifest. 16

## System map

The existing system is broad, but the hero→overview path is concentrated in a manageable set of files.

| File                                                 | Responsibility                                                                                                                                                                                                                                                      | Why it matters for hero→overview                                                                                                            |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/App.jsx</code>                             | Owns editable <code>config</code> , rebuilds <code>animationConfig</code> with <code>buildAnimationConfig</code> , computes runtime overrides, and routes nav clicks to <code>Fixed3DCanvas</code> direct zone selection. <span>10</span>                           | This is the runtime config funnel. Any new cinematic tuning must flow through <code>config</code> → <code>buildAnimationConfig</code> here. |
| <code>src/components/layout/Fixed3DCanvas.jsx</code> | Merges layout + runtime overrides into <code>cameraMergedConfig</code> , mounts <code>UnifiedCameraController</code> and <code>UnifiedCrystalScene</code> , shares camera move progress ref, and exposes imperative <code>directSelectZone</code> . <span>11</span> | This is the best insertion point for a shared hero→overview runtime or ref, because both camera and scene already meet here.                |

| File                                                             | Responsibility                                                                                                                                                                                                                                                                                                                                            | Why it matters for hero→overview                                                                                                                                        |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/components/three/MasterAnimationCoordinator.jsx</code> | Combines <code>useScrollProgress</code> and <code>useUnifiedAnimationController</code> , then passes <code>animationData</code> plus direct select callbacks down to children. In the phase branch it also passes <code>transitionPhase</code> , <code>setTransitionPhase</code> , camera-settled APIs, and config outputs. <sup>5</sup>                  | This is where a “transition request” should originate, but <b>not</b> where per-frame cinematic ownership should live.                                                  |
| <code>src/hooks/useScrollProgress.js</code>                      | Tracks scroll progress, snapping, current section, and exposes <code>scrollToProgress</code> / <code>scrollToZone</code> . <sup>17</sup>                                                                                                                                                                                                                  | Hero→overview is still fundamentally initiated by section movement or direct section jumps.                                                                             |
| <code>src/hooks/useUnifiedAnimationController.js</code>          | Base animation state machine for zone/project state, camera config, crystal config, direct-project/zone overrides. In PR #381 it adds <code>HERO_TO_OVERVIEW_PHASES</code> , mutable <code>transitionPhase</code> , <code>setTransitionPhase</code> , <code>forceTransitionPhase</code> , and <code>heroToOverviewExplosionSettings</code> . <sup>2</sup> | This is currently trying to do too much. It should keep high-level zone/project state, but the hero→overview cinematic should stop being a multi-writer phase API here. |
| <code>src/components/three/UnifiedCameraController.jsx</code>    | Owns per-frame camera mutation via <code>useFrame</code> , hero orbit, project target resolution, forced hero↔overview transitions, fracture tilt, handoff locks, and transition refs. <sup>4</sup>                                                                                                                                                       | This is the current camera ownership hot zone and the biggest freeze risk.                                                                                              |
| <code>src/components/three/UnifiedCrystalScene.jsx</code>        | Owns whole/facet visibility, fracture/explode/reform swaps, facet transforms, labels, hover/material behavior, particles, and overview anchor behavior. In PR #381 it also starts phase changes from scene-side events. <sup>3 18</sup>                                                                                                                   | This is the current facet/visibility ownership hot zone.                                                                                                                |
| <code>src/components/ui/CrystalControls.jsx</code>               | Control panel state, config cloning, layout payload export/import, and per-device editing based on active desktop/mobile device. It already builds <code>schemaVersion: 2</code> payloads from runtime config and applies layout payloads back into config. <sup>7</sup>                                                                                  | This is where new cinematic tuning controls belong.                                                                                                                     |

| File                                                                   | Responsibility                                                                                                                                                                                                  | Why it matters for hero→overview                                                          |
|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <code>src/hooks/useLayoutConfig.js</code>                              | Selects desktop or mobile layout by media query and parses it. <sup>8</sup>                                                                                                                                     | This already honors device-specific layout JSON; keep that contract.                      |
| <code>src/lib/layout/parseLayout.js</code>                             | Validates <code>schemaVersion: 2</code> payloads and currently recognizes anchors, camera positions/targets/offsets/projects/composition/heroTuning, and project layout fields. <sup>9</sup>                    | This is the explicit schema extension point for new cinematic tuning.                     |
| <code>src/config/layout/desktop.json</code> / <code>mobile.json</code> | Current per-device authored layout payloads, including camera positions, targets, offsets, project camera settings, composition, hero tuning, project positions, and timing.camera. <sup>12</sup> <sup>13</sup> | These must remain the canonical live-edit/export format.                                  |
| <code>src/data/projects.js</code>                                      | Maps project ids, facet keys, placement keys, and scene facet keys. <sup>19</sup>                                                                                                                               | Any scene/controller that refers to “project vs scene facet key” depends on this mapping. |

The JSON/export path is already coherent. `CrystalControls` imports both `desktop.json` and `mobile.json`, detects the active device, builds a layout payload from current config, and can apply a layout payload back into runtime config. `useLayoutConfig` selects the correct device JSON at runtime, and `parseLayout` enforces structural validity. `App.jsx` then turns the edited config back into runtime camera/crystal vectors and quaternions through `buildAnimationConfig`. That means you do **not** need a new settings system for this transition; you only need to extend the existing one in a backward-compatible way. <sup>7</sup> <sup>8</sup> <sup>9</sup> <sup>10</sup>

A useful contextual note: PR #365 proposed a more formal camera tuning payload/validation layer, but it is not merged into the current repo. It is relevant as inspiration, not as present-state infrastructure. For the plan below, I would **not** make PR #365 a dependency; I would keep using the current layout JSON path and only borrow ideas later if needed. <sup>14</sup>

# Ownership audit and current failure points

## Ownership audit

| Animated resource                                                                                                             | Current writers                                                                                                                                                                                                                                                                                                    | Gates / refs / conditions                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>camera.position</code> ,<br><code>camera.lookAt</code> ,<br><code>camera.filmOffset</code> ,<br><code>camera.fov</code> | Primarily<br><code>UnifiedCameraController</code> , which<br>contains hero orbit, intro, forced<br>hero→overview / overview→hero<br>transitions, fracture tilt seeding/lock,<br>project focus target locking, and<br>normal smoothing. 4                                                                           | <code>fractureTiltActiveRef</code> ,<br><code>heroExplosionTransitionRef</code> ,<br><code>authoritativeHeroToOverviewTransitionRef</code> ,<br><code>authoritativeOverviewToHeroTransitionRef</code> ,<br>handoff refs, plus <code>animationData.cameraState/<br/>state/focusedFacet</code> . 4 |
| Facet transforms and<br>facet rotations                                                                                       | <code>UnifiedCrystalScene</code> via explode/<br>reform swap functions, fracture/<br>explosion timers, focus/project/<br>overview positioning, and per-frame<br>mutation of refs. 18 3                                                                                                                             | <code>showWholeCrystal</code> , <code>showFacets</code> ,<br><code>explosionStartRef</code> ,<br><code>pendingExplodeSwapAtRef</code> ,<br><code>pendingReformSwapAtRef</code> ,<br><code>fractureChargeActiveRef</code> , focus state, project<br>state. 3                                      |
| Whole crystal / facets /<br>sphere / ring visibility                                                                          | <code>UnifiedCrystalScene</code> local state<br>and swap helpers. 18 3                                                                                                                                                                                                                                             | Local <code>useState</code> plus pending swap refs and<br>crystal-form transitions. 3                                                                                                                                                                                                            |
| Burst particles and<br>glow masking                                                                                           | <code>UnifiedCrystalScene</code> increments<br>burst ids and glow/mask refs; PR #342<br>also formalized the low-risk glow-mask<br>approach. 3 6                                                                                                                                                                    | Burst ids, fracture glow timers, swap mask timers,<br>visibility swaps. 3                                                                                                                                                                                                                        |
| Transition phase /<br>cinematic status                                                                                        | <code>useUnifiedAnimationController</code><br>owns mutable <code>transitionPhase</code> ;<br><code>UnifiedCrystalScene</code> also<br>advances it from swap/progress<br>events;<br><code>MasterAnimationCoordinator</code><br>passes it to children;<br><code>UnifiedCameraController</code><br>observes it. 2 3 5 | <code>setTransitionPhase</code> ,<br><code>forceTransitionPhase</code> , direct zone overrides,<br>overview completion hooks. 2                                                                                                                                                                  |

|                     |                                                                                                          |                                                                                                                                 |
|---------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| JSON tuning payload | <pre>CrystalControls, useLayoutConfig, parseLayout, desktop/mobile.json, App.buildAnimationConfig.</pre> | Optional runtime override merging in <code>Fixed3DCanvas</code> , config editing in <code>App</code> , current schemaVersion 2. |
|---------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|

## What is actually failing

The transition branch in PR #381 added a new lifecycle whose intent is reasonable: named phases for `fractureCharge`, `explosionImpulse`, `bulletTimeSlowdown`, `overviewHandoff`, and `complete`, plus a bundle of `heroToOverviewExplosionSettings` tuning values. But the branch leaves the older camera and scene ownership machinery in place. That means `transitionPhase` is **observationally valuable**, but it is not the only thing that decides who writes camera and facets. 1 2 3 4

That is why the current debugging loop has felt endless. The branch keeps adding good diagnostics to a fundamentally split ownership model. Even the repo history reflects this tension: PR #342 explicitly framed the safest path as a surgical visual mask that avoided a large new transition state machine, while PR #379 later landed a very small camera-orientation capture fix using the live camera transform via `camera.getWorldDirection`, which is exactly what Three.js documents that API for. 6 15 20

There is a second integration risk hiding beneath the freeze itself: the existing export/import path does **not** yet understand hero→overview cinematic tuning. `parseLayout` currently validates camera positions, targets, offsets, project-specific camera data, camera composition, hero tuning, and project geometry-related layout data, but there is no defined hero→overview cinematic block in the current schema. `App.buildAnimationConfig` rebuilds camera vectors and crystal transforms from edited config, but it likewise does not yet map a cinematic block through to runtime. If you change the transition in code without wiring this path, you will create yet another “hidden tuning island.” 9 10

## Minimal controller design

### The design in one sentence

Create one **HeroOverviewCinematicTransition** runtime that is the single source of truth for the hero→overview cinematic, make phase **derived from elapsed time** instead of mutable from multiple components, and let camera and scene each have **one** gated cinematic writer branch while the runtime is active.

## Where it should live

The cleanest minimal insertion point is **inside the Canvas tree but above both camera and scene**, with the shared runtime object created in `Fixed3DCanvas` and passed into both `UnifiedCameraController` and `UnifiedCrystalScene`. `MasterAnimationCoordinator` should only raise a **request** (“start hero→overview”), not own the cinematic timeline itself. That fits the current composition of `Fixed3DCanvas` and `MasterAnimationCoordinator` without blowing up the rest of the app. 11 5

## Ownership rules

The rule set needs to be brutally simple:

1. **When** `heroOverviewRuntime.active === false`, the app behaves exactly as it does now.
2. **When** `heroOverviewRuntime.active === true`, `UnifiedCameraController` may only run its dedicated hero→overview cinematic camera branch, and `UnifiedCrystalScene` may only run its dedicated hero→overview cinematic facet/visibility branch.
3. No other branch may write camera or facet transforms during that window.
4. `phase` becomes **derived** from elapsed time inside the runtime, not set by scene or camera.
5. Release back to normal overview behavior happens in exactly one place: the runtime’s `complete → release` handoff.

This matches React Three Fiber guidance: per-frame animation should live in `useFrame`, driven by mutable refs, without pushing fast-changing transient values through React state on every frame. It also aligns with React’s intended use of `useRef` for values that are not needed for rendering. 21

## Proposed API

Use a new file, for example:

```
// src/components/three/HeroOverviewCinematicTransition.js
export function createHeroOverviewRuntime(defaults) {
  return {
    requestId: 0,
    request: null,          // { id, source, fromZone, toZone }
    active: false,
    startedAt: 0,
    seed: null,             // camera + whole/facet snapshots
    snapshot: null,         // derived frame snapshot
    settings: defaults,     // merged runtime tuning
    requestStart(payload) {},
    begin(seedPayload) {},
    update(nowSeconds) {},
    release(reason) {},
    abort(reason) {},
  }
}
```

```

    }
}

```

The runtime should produce a **derived snapshot** like:

```

type HeroOverviewSnapshot = {
  active: boolean
  requestId: number
  phase: 'idle' | 'fractureCharge' | 'explosionImpulse' | 'bulletTimeSlowdown'
  | 'overviewHandoff' | 'complete'
  elapsedMs: number
  progress: number
  camera: {
    position: Vector3
    lookAt: Vector3
    filmOffset: number
    fov: number
  }
  scene: {
    wholeVisible: boolean
    facetsVisible: boolean
    sphereVisible: boolean
    ringVisible: boolean
    facetPoses: Record<string, { position: Vector3, quaternion: Quaternion }>
    triggerBurst: boolean
    triggerLeak: boolean
  }
}

```

## Phase derivation

Keep the names you already introduced in PR #381, but stop mutating them from multiple places. Reuse the current parameter vocabulary where possible so you do not have to re-tune from scratch. PR #381 already defines useful knobs like `fractureChargeDuration`, `fractureSpread`, `fractureJitterStrength`, `explosionImpulseDuration`, `explosionImpulseDistance`, `explosionImpulseStrength`, `explosionCameraPushbackDistance`, `explosionCameraPushbackStrength`, `bulletTimeSlowdownDuration`, and `overviewHandoffDuration`. Those names are worth keeping. <sup>2</sup>

A derived-phase helper is enough:

```

function derivePhase(elapsedMs, d) {
  if (elapsedMs < d.fractureChargeMs) return 'fractureCharge'
  if (elapsedMs < d.fractureChargeMs + d.impulseMs) return 'explosionImpulse'
  if (elapsedMs < d.fractureChargeMs + d.impulseMs + d slowdownMs) return

```



```
'bulletTimeSlowdown'
  if (elapsedMs < d.totalMs) return 'overviewHandoff'
  return 'complete'
}
```

No `setTransitionPhase`. No `forceTransitionPhase`. No scene-side phase advancing. No camera-side phase fixing. Just derivation.

## Suggested timeline

The current repo already contains the ingredients for a strong cinematic: a masked whole→facets swap in scene code, fracture glow behavior, direct snap-to-fracture positions, and a camera-side live-look-at capture fix via `camera.getWorldDirection`. Three.js explicitly documents `Camera.getWorldDirection` as the world-space look direction of the camera, and `PerspectiveCamera` requires `updateProjectionMatrix()` after `filmOffset` or related projection changes. 3 15 20

Use that to structure the timeline:

```
flowchart LR
  A[hero zone request] --> B[requestStart]
  B --> C[seed live camera + whole crystal pose]
  C --> D[fractureCharge]
  D --> E[hidden whole-facets swap]
  E --> F[explosionImpulse]
  F --> G[bulletTimeSlowdown]
  G --> H[overviewHandoff]
  H --> I[release to normal overview]
```

A more concrete runtime handoff:

```
sequenceDiagram
    participant Coordinator as MasterAnimationCoordinator
    participant Runtime as HeroOverviewRuntime
    participant Camera as UnifiedCameraController
    participant Scene as UnifiedCrystalScene

    Coordinator->>Runtime: request(hero -> overview)
    Camera->>Runtime: seed live camera snapshot
    Scene->>Runtime: seed fracture/facet snapshot
    loop each frame while active
        Runtime-->>Camera: derived camera pose
        Runtime-->>Scene: derived visibility + facet poses
    end
    Runtime->>Coordinator: complete/released
```

## Camera behavior in the new model

Instead of keeping pushback hidden behind a fracture-tilt lock branch, bake it directly into the cinematic camera curve:

- During `fractureCharge`, largely hold the current hero camera, optionally with a tiny tightening parameter.
- At `explosionImpulse`, perform the hidden representation swap and apply pushback directly along the camera's backward vector derived from the live camera snapshot.
- During `bulletTimeSlowdown`, move from the impulse endpoint toward the overview target with a slower easing curve.
- During `overviewHandoff`, settle onto the authored overview camera and zero any temporary film offset / transient framing changes.
- Then release.

That preserves the visual intent of PR #381 but removes the dependency on legacy fracture-tilt ownership refs as the effective camera owner. 2 4

## Scene behavior in the new model

`UnifiedCrystalScene` should keep ownership of scene-local concerns, but only under the new runtime snapshot:

- `fractureCharge`: whole crystal visible, glow/mask pre-charge, no visible facet motion yet.
- boundary into `explosionImpulse`: perform one hidden whole→facets swap and fire burst/leak one-shots.
- `explosionImpulse` / `bulletTimeSlowdown`: facets follow the runtime's derived positions and quaternions.
- `overviewHandoff`: facets ease into authored exploded overview positions exactly once.
- after `release`: normal overview float/labels/hover/project selection resume.

This preserves the successful scene-local parts of the current implementation instead of rewriting everything. PR #342 already established that a local scene-side masking strategy is viable and low-risk. 3

6

## Control panel and export integration

The current schema does **not** specify hero→overview cinematic settings, so I recommend a **backward-compatible optional block** added to the existing layout payload. The least disruptive option is to keep `schemaVersion: 2` and add an optional top-level block such as:

```
{
  "schemaVersion": 2,
  "heroOverview": {
    "durations": {
      "fractureChargeMs": 120,
      "explosionImpulseMs": 250,
```

```

    "bulletTimeSlowdownMs": 850,
    "overviewHandoffMs": 500
  },
  "camera": {
    "pushbackDistance": 0.55,
    "pushbackStrength": 1.0,
    "lookAtTightening": 0.0
  },
  "facets": {
    "fractureSpread": 0.08,
    "fractureJitterStrength": 0.02,
    "explosionImpulseDistance": 0.32,
    "explosionImpulseStrength": 1.35,
    "rotationStrength": 1.15
  },
  "particles": {
    "burstStrength": 1.0,
    "leakStrength": 0.2,
    "glowPeakIntensity": 2.3
  }
}

```

Why this shape:

- It stays device-specific because it lives in desktop/mobile JSON just like the existing camera layout.
- It does not break old saved payloads because the block is optional.
- It keeps camera and facet tuning in one place instead of splitting them between multiple hidden config sources.
- It avoids depending on unmerged PR #365. [7](#) [9](#) [12](#) [13](#) [14](#)

The file insertion points are straightforward:

- `parseLayout.js`: recognize optional `heroOverview`.
- `useLayoutConfig.js`: unchanged, because it already pipes the parsed layout through.
- `CrystalControls.jsx`: add UI state + export/import support for `heroOverview`.
- `App.buildAnimationConfig`: map `config.heroOverview` into runtime animation settings.
- `Fixed3DCanvas.jsx`: pass resulting settings into the shared `hero→overview` runtime.
- `UnifiedCameraController.jsx` and `UnifiedCrystalScene.jsx`: consume the runtime snapshot, not mutable phase setters. [9](#) [7](#) [10](#) [11](#)

## Migration plan

### Branch strategy

Do **not** continue building on PR #381. Create a fresh branch from the last stable visual baseline. If the last stable base is `master`, branch from `master`. Then selectively cherry-pick only extremely isolated fixes

that have clear value, such as the live-look-at capture approach from PR #379 if you still need it. PR #379 is small, focused, and aligned with Three.js camera semantics; PR #381 is not a good base for continued layering. 15 1 20

Commit-sized implementation sequence

| Commit             | Goal                                                                                                                                    | Files                                                                                                 | Expected logs                                                                                                                                                              | Validation                                  |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| Baseline reset     | Create fresh branch from stable baseline; no behavior change.                                                                           | none or minimal docs                                                                                  | none                                                                                                                                                                       | npm run build                               |
| Runtime scaffold   | Add <code>HeroOverviewRuntime</code> as a no-op shared ref/runtime and request plumbing only. Do <b>not</b> change visual behavior yet. | <code>Fixed3DCanvas.jsx</code> ,<br><code>MasterAnimationCoordinator.jsx</code> ,<br>new runtime file | <code>[hero-overview-transition] request</code> ,<br><code>[hero-overview-transition] aborted</code>                                                                       | Build + check scroll smooth test            |
| Camera integration | Add one gated cinematic camera branch to <code>UnifiedCameraController</code> ; if runtime is inactive, old behavior remains unchanged. | <code>UnifiedCameraController.jsx</code> ,<br>runtime file                                            | <code>[hero-overview-transition] seeded</code> ,<br><code>[hero-overview-transition] camera-owner-acquired</code> ,<br><code>[hero-overview-transition] phase-enter</code> | Build + hero→overview scroll and Work click |

| Commit                     | Goal                                                                                                                                                          | Files                                                                                                                                                                         | Expected logs                                                                                                                                                     | Validation                                    |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Scene integration          | Add one gated cinematic scene branch to <code>UnifiedCrystalScene</code> ; scene follows runtime snapshot, and old scene logic stays untouched when inactive. | <code>UnifiedCrystalScene.jsx</code> , runtime file                                                                                                                           | <code>[hero-overview-transition] facet-owner-acquired</code> , <code>[hero-overview-transition] swap</code> , <code>[hero-overview-transition] burst-fired</code> | Build + hero→ov scroll, ve freeze an snap-bac |
| Export/control integration | Add optional <code>heroOverview</code> payload block to live config, export/import, and parser.                                                               | <code>CrystalControls.jsx</code> , <code>parseLayout.js</code> , <code>desktop.json</code> , <code>mobile.json</code> , <code>App.jsx</code>                                  | <code>[hero-overview-transition] config-applied</code>                                                                                                            | Build + e import d and mob payloads           |
| Cleanup                    | Remove <code>transitionPhase</code> writes from camera/scene flow, keep at most a read-only compatibility shim for one pass. Remove old debug spam.           | <code>useUnifiedAnimationController.js</code> , <code>MasterAnimationCoordinator.jsx</code> , <code>UnifiedCameraController.jsx</code> , <code>UnifiedCrystalScene.jsx</code> | <code>[hero-overview-transition] released</code>                                                                                                                  | Build + fu manual regressio                   |
| Final hardening            | Add final one-shot diagnostics and feature-flag cleanup.                                                                                                      | same as above                                                                                                                                                                 | <code>[hero-overview-transition] complete only</code>                                                                                                             | Repeated hero↔ov project, c study, ab tests   |

## Exact debug logs to add

Use one-shot logs only. Do **not** log every frame.

```
[hero-overview-transition] request
[hero-overview-transition] seeded
[hero-overview-transition] camera-owner-acquired
[hero-overview-transition] facet-owner-acquired
[hero-overview-transition] phase-enter
[hero-overview-transition] swap
[hero-overview-transition] burst-fired
[hero-overview-transition] handoff
[hero-overview-transition] released
```

```
[hero-overview-transition] aborted  
[hero-overview-transition] config-applied
```

Example payloads:

```
console.log('[hero-overview-transition] request', {  
  requestId,  
  source: 'scroll' | 'direct-zone',  
  fromZone: 'hero',  
  toZone: 'overview'  
})  
  
console.log('[hero-overview-transition] seeded', {  
  requestId,  
  startedAt,  
  cameraFrom: cameraSnapshot.position.toArray(),  
  cameraTo: overviewTarget.position.toArray(),  
  facetCount: 6  
})  
  
console.log('[hero-overview-transition] phase-enter', {  
  requestId,  
  phase: snapshot.phase,  
  elapsedMs: snapshot.elapsedMs  
})  
  
console.log('[hero-overview-transition] released', {  
  requestId,  
  reason: 'complete'  
})
```

Old diagnostics to remove after stabilization:

```
[scene-freeze-debug]  
[transition-phase-debug]  
[explosion-camera-debug]  
[visual-owner-debug]  
[freeze-root-debug]  
[stability-debug]
```

## Build and test commands

The current repo exposes only these scripts:

```
npm install
npm run build
npm run preview
```

There is no current `npm test` or `npm run lint` script in `package.json`, so browser QA is part of the primary validation loop. <sup>16</sup>

## Manual QA checklist

Run all of these after every meaningful step:

- Hero → Overview by scroll
- Hero → Work by navigation click
- Hero → Work → Hero → Work repeated several times
- Overview → Hero reform
- Overview → Project → Case study → Back
- About path unchanged
- Desktop export/import round-trip
- Mobile export/import round-trip

Those flows matter because the current app routes direct zone changes from `App` into `Fixed3DCanvas`, and the Canvas then passes animation/control data down through `MasterAnimationCoordinator`, `UnifiedCameraController`, and `UnifiedCrystalScene`. <sup>10</sup> <sup>11</sup> <sup>5</sup>

## Risks, rollback, and exact Codex prompts

### Risk assessment

The biggest risk is not the new controller itself. The biggest risk is **incomplete gating**, where some old camera or scene branch still writes during the new cinematic window. That is why the first implementation goal must be ownership, not tuning. If the new runtime is active and *any* legacy camera/facet branch still writes, you will recreate the same class of problems under a different name. The repo's current structure makes that risk very real. <sup>4</sup> <sup>3</sup>

The second major risk is config drift. Because the app already has a coherent desktop/mobile JSON workflow, it would be a mistake to add transition tuning directly into code without putting it into `CrystalControls`, `parseLayout`, and `App.buildAnimationConfig`. That would recreate the very "over-engineered hidden system" problem you are trying to get out of. <sup>7</sup> <sup>9</sup> <sup>10</sup>

The third risk is trying to refactor both hero→overview and overview→hero at the same time. Do not do that. Keep overview→hero exactly as-is until hero→overview is stabilized. The current code already has separate transition machinery in camera and scene for multiple directions; trying to "clean all of it at once" is how this effort balloons again. <sup>4</sup> <sup>3</sup>

## Rollback plan

Use a feature flag for the new runtime during integration:

```
const ENABLE_HERO_OVERVIEW_RUNTIME = true
```

If a step goes bad, rollback is clean:

- Disable the feature flag to return to baseline behavior.
- Revert the last commit only.
- Do **not** revert the whole branch unless the scaffold itself is wrong.
- Keep PR #381 open and untouched as a reference branch only.

This is why the commit order above is important: each step should be independently reversible.

## Exact Codex prompts

Use these prompts in a **new Codex chat** on a **fresh branch**. Do not continue the current PR #381 loop.

### Prompt for the scaffold step

Work on a fresh branch from the current stable baseline. Do not modify the open PR #381 branch.

Goal: add a new no-op shared runtime for the hero -> overview cinematic, without changing visual behavior yet.

Requirements:

- Create a new runtime module named HeroOverviewRuntime (or equivalent) that can hold:
  - request state
  - active flag
  - start time
  - seeded camera snapshot
  - seeded facet snapshot
  - derived snapshot placeholder
- Instantiate the runtime in Fixed3DCanvas and pass it to UnifiedCameraController and UnifiedCrystalScene.
- In MasterAnimationCoordinator, emit only a transition request for hero -> overview and direct overview selection. Do NOT add new visual behavior.
- Add one-shot debug logs:
  - [hero-overview-transition] request
  - [hero-overview-transition] aborted
- Do not change Overview -> Hero behavior.



- Do not remove existing systems yet.
- Run npm run build and report any errors.

### Prompt for the camera ownership step

Using the new shared HeroOverviewRuntime already added, implement camera ownership for hero -> overview only.

Requirements:

- In UnifiedCameraController, add exactly one new gated branch:
  - if heroOverviewRuntime.active === true, only the new cinematic camera path may write camera.position, camera.lookAt, camera.filmOffset, or camera.fov
  - all legacy camera branches must early-out while the runtime owns the transition
- Seed the runtime from the live camera transform, using camera.getWorldDirection for the live look direction fallback
- Keep filmOffset / projection updates correct by calling updateProjectionMatrix after projection-affecting changes
- Do not touch scene/facet behavior yet except whatever is needed to seed the runtime
- Add one-shot logs:
  - [hero-overview-transition] seeded
  - [hero-overview-transition] camera-owner-acquired
  - [hero-overview-transition] phase-enter
- Do not add per-frame spam logs
- Preserve all non hero->overview paths unchanged
- Run npm run build

### Prompt for the scene ownership step

Using the shared HeroOverviewRuntime and the new camera owner branch, implement scene ownership for hero -> overview only.

Requirements:

- In UnifiedCrystalScene, add exactly one new gated cinematic branch:
  - if heroOverviewRuntime.active === true, only the new cinematic scene path may write facet transforms or transition visibility flags
  - old hero->overview scene branches must early-out while active
- Keep whole crystal / facets / sphere / ring behavior visually equivalent to the current intended flow
- Preserve glow masking / burst behavior as much as possible, but route timing from the runtime snapshot instead of local phase writes
- Do not change Overview -> Hero reform behavior
- Add one-shot logs:
  - [hero-overview-transition] facet-owner-acquired

- [hero-overview-transition] swap
- [hero-overview-transition] burst-fired
- [hero-overview-transition] released
- Remove no old debug logs yet
- Run npm run build

### Prompt for the export and control-panel step

Extend the existing desktop/mobile layout JSON workflow to support hero -> overview cinematic tuning, without breaking old saved configs.

Requirements:

- Preserve schemaVersion: 2 for now
- Add an optional top-level heroOverview block to the layout schema and parser
- Update CrystalControls to edit/export/import this block
- Update App.buildAnimationConfig so the exported/imported heroOverview values are applied to runtime animation config
- Keep backward compatibility: if heroOverview is absent, current defaults must still work
- Do not introduce a new independent settings system
- Add one-shot log:
  - [hero-overview-transition] config-applied
- Run npm run build

### Prompt for the cleanup step

Now remove the old overlapping hero -> overview phase ownership, but only after the new runtime path is working.

Requirements:

- Remove mutable hero->overview phase writes from camera and scene flow
- Keep only a read-only derived phase in the new runtime if needed for debug or UI
- Remove forceTransitionPhase / setTransitionPhase usage from the hero->overview cinematic path
- Keep zone/project state logic in useUnifiedAnimationController intact for the rest of the app
- Remove old debug logs related to freeze hunting:
  - [scene-freeze-debug]
  - [transition-phase-debug]
  - [explosion-camera-debug]
  - [visual-owner-debug]
  - [freeze-root-debug]
  - [stability-debug]

- Keep final one-shot runtime logs only
- Run npm run build

## Open questions and limitations

The main open question is not architectural; it is **which commit you personally regard as the last visually stable baseline** for hero→overview. This report strongly recommends starting from that baseline rather than from PR #381, but the repo alone cannot tell me which branch you subjectively trust most in-browser. The plan remains the same either way. <sup>1</sup>

A second limitation is that some very large file fetches were truncated in the connector response, so a few deeper branch details were validated through PR metadata and visible code structure rather than complete full-file dumps. I have called out uncertainty where it matters and avoided claiming certainty where the evidence was incomplete.

---

### <sup>1</sup> Add transition phase tracking and hero-to-overview phase hooks

<https://github.com/jjonny00/crystal-portfolio/pull/381>

### <sup>2</sup> useUnifiedAnimationController.js

<https://github.com/jjonny00/crystal-portfolio/blob/codex/audit-crystal-transition-animation-code/src/hooks/useUnifiedAnimationController.js>

### <sup>3</sup> UnifiedCrystalScene.jsx

<https://github.com/jjonny00/crystal-portfolio/blob/codex/audit-crystal-transition-animation-code/src/components/three/UnifiedCrystalScene.jsx>

### <sup>4</sup> UnifiedCameraController.jsx

<https://github.com/jjonny00/crystal-portfolio/blob/codex/audit-crystal-transition-animation-code/src/components/three/UnifiedCameraController.jsx>

### <sup>5</sup> MasterAnimationCoordinator.jsx

<https://github.com/jjonny00/crystal-portfolio/blob/codex/audit-crystal-transition-animation-code/src/components/three/MasterAnimationCoordinator.jsx>

### <sup>6</sup> Refine crystal transition glow masking

<https://github.com/jjonny00/crystal-portfolio/pull/342>

### <sup>7</sup> CrystalControls.jsx

<https://github.com/jjonny00/crystal-portfolio/blob/master/src/components/ui/CrystalControls.jsx>

### <sup>8</sup> useLayoutConfig.js

<https://github.com/jjonny00/crystal-portfolio/blob/master/src/hooks/useLayoutConfig.js>

### <sup>9</sup> parseLayout.js

<https://github.com/jjonny00/crystal-portfolio/blob/master/src/lib/layout/parseLayout.js>

### <sup>10</sup> App.jsx

<https://github.com/jjonny00/crystal-portfolio/blob/codex/audit-crystal-transition-animation-code/src/App.jsx>

### <sup>11</sup> Fixed3DCanvas.jsx

<https://github.com/jjonny00/crystal-portfolio/blob/codex/audit-crystal-transition-animation-code/src/components/layout/Fixed3DCanvas.jsx>

12 **desktop.json**

<https://github.com/jjonny00/crystal-portfolio/blob/master/src/config/layout/desktop.json>

13 **mobile.json**

<https://github.com/jjonny00/crystal-portfolio/blob/master/src/config/layout/mobile.json>

14 **Introduce camera tuning payload, validation, and integration into camera/config flow**

<https://github.com/jjonny00/crystal-portfolio/pull/365>

15 **Fix camera jump/tilt during hero ↔ overview transitions**

<https://github.com/jjonny00/crystal-portfolio/pull/379>

16 **package.json**

<https://github.com/jjonny00/crystal-portfolio/blob/master/package.json>

17 **useScrollProgress.js**

<https://github.com/jjonny00/crystal-portfolio/blob/master/src/hooks/useScrollProgress.js>

18 **UnifiedCrystalScene.jsx**

<https://github.com/jjonny00/crystal-portfolio/blob/master/src/components/three/UnifiedCrystalScene.jsx>

19 **projects.js**

<https://github.com/jjonny00/crystal-portfolio/blob/master/src/data/projects.js>

20 **<https://threejs.org/docs/pages/Camera.html>**

<https://threejs.org/docs/pages/Camera.html>

21 **<https://r3f.docs.pmnd.rs/tutorials/basic-animations>**

<https://r3f.docs.pmnd.rs/tutorials/basic-animations>